- It’s generally easier to test FluentValidation validators than `DataAnnotations` attributes.
- The validation is strongly typed compared with `DataAnnotations` attributes where it’s possible to apply attributes in ways that don’t make sense, such as applying an `[EmailAddress]` attribute to an `int` property.
- Separating your validation logic from the model itself arguably better conforms to the single-responsibility principle (SRP).

That final point is sometimes given as a reason not to use FluentValidation: FluentValidation separates a binding model from its validation rules. Some people are happy to accept the limitations of `DataAnnotations` to keep the model and validation rules together.

Before I show how to add FluentValidation to your application, let’s see what FluentValidation validators look like.

32.4.1 Comparing FluentValidation with DataAnnotations attributes

To better understand the difference between the `DataAnnotations` approach and FluentValidation, we’ll convert the binding models from section 32.3 to use FluentValidation. The following listing shows what the binding model from listing 32.7 would look like when used with FluentValidation. It is structurally identical but has no validation attributes.

Listing 32.10 Currency converter initial binding model for use with FluentValidation

```
public class CurrencyConverterModel
{
```